



Advanced data-cleaning functions, part 1

Hi there and welcome back. So far we've gone over some basic SQL queries and functions that can help you clean your data. We've also checked out some ways you can deal with string variables in SQL to make your job easier. Get ready to learn more functions for dealing with strings in SQL. Trust me, these functions will be really helpful in your work as a data analyst. In this video, we'll check out strings again and learn how to use the cast function to correctly format data. When you import data that doesn't already exist in your SQL tables, the data types from the new dataset might not have been imported correctly.

This is where the CAST function comes in handy. Basically, CAST can be used to convert anything from one data type to another. Let's check out an example. Imagine we're working with Lauren's Furniture Store. The owner has been collecting transaction data for the past year, but she just discovered that they can't actually organize their data because it hadn't been formatted correctly. So we'll help her by converting her data to make it useful again. For example, let's say we want to sort all purchases by `purchase_price` in descending order.

That means we want the most expensive purchase to show up first in our results. To write the SQL query, we start with the basic SQL structure. SELECT, FROM, WHERE, we know the data is stored in the `customer_purchase` table in the `customer_dataset`. So

we write `customer_data.customer_purchase` after `FROM`. Next, we tell SQL what data to give us in the select clause. We want to see the `purchase_price` data, so we type `purchase_price` after `SELECT`. Next is the where clause.

We are not filtering out any data since we want all purchase prices shown, so we can take out the where clause. Finally, to sort the `purchase_price` in descending order, we type `ORDER BY purchase_price DESC` at the end of our query. Let's run this query. We see that 89.85 shows up at the top with 799.99 below it, but we know that 799.99 is a bigger number than 89.85. The database doesn't recognize that these are numbers, so it didn't sort them that way. If we go back to the `customer_purchase` table and take a look at its schema, we can see what data type the database thinks `purchase_price` is. It says here the database thinks `purchase_price` is a string, when in fact it is a float, which is a number that contains a decimal.

That is why 89.85 shows up before 799.99. When we sort letters, we start from the first letter before moving on to the second letter. So if we want to sort the words apple and orange in descending order, we start with the first letters a and o. Since o comes after a, orange will show up first, then apple. The database did the same with 89.85 and 799.99. It started with the first letter, which in this case was 8 and 7 respectively. Since 8 is bigger than 7, the database sorted 89.85 first and then 799.99 because the database treated these as text strings.

The database doesn't recognize these strings as floats because they haven't been typecast to match that data type yet. Typecasting means converting data from one type to another, which is what we'll do with the `CAST` function. We use the `CAST` function to replace `purchase_price` with a new `purchase_price` that the database recognizes as

float instead of string. We start by replacing `purchase_price` with `CAST`. Then we tell SQL the field we want to change, which is the `purchase_price` field. Next is a data type we want to change `purchase_price` to, which is the `FLOAT` data type. BigQuery stores numbers in a 64 bit system, so the `FLOAT` data type is referenced as `float 64` in our query.

This might be slightly different in other SQL platforms, but basically the `64` and `float 64` just indicates that we're casting numbers in the 64 bit system as `FLOATs`. We also need to sort this new field so we change `purchase_price` after `ORDER BY` to `CAST purchase_price as FLOAT64`. This is how we use the cast function to allow SQL to recognize the `purchase_price` column as `FLOATs` instead of text strings. Now we can sort our purchases by `purchase_price`. And just like that, Lauren's Furniture Store has data that can actually be used for analysis. As a data analyst, you'll be asked to locate and organize data a lot, which is why you want to make sure you convert between data types early on. Businesses like our Furniture Store are interested in timely sales data, and you need to be able to account for that in your analysis.

The `CAST` function can be used to change strings into other data types too, like date and time. As a data analyst, you might find yourself using data from various sources. Part of your job is making sure the data from those sources is recognizable and usable in your database so that you won't run into any issues with your analysis. And now you know how to do that. The `CAST` function is one great tool you can use when you're cleaning data. And coming up, we'll cover some other advanced functions that you can add to your toolbox. See you soon.